RAMDEOBABA UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



B. TECH PROJECT REPORT

B.Tech. in Computer Science and Engineering
(Data Science and Analytics)

ON

CYBERSHIELD - REAL-TIME CYBER ATTACK DETECTION

Submitted by:

Aashi Kothale (Roll No. B4-B1-01)

Suhani Parate (Roll No. B4-B2- 25)

Trupti Avhadkar (Roll No. B4- B2-26)

Under the Guidance of :

Prof. Vinita Kakani

Course Code: 24CS04TH0501

Academic Year: 2026–2027

ABSTRACT

CyberShield is a browser-based academic prototype for exploring network intrusion detection through a cybersecurity dashboard. The supplied implementation contains 10,000 embedded network-event records and replays them at a configurable interval. Its interface provides a dashboard, live-monitoring table, attack-detection panel, attack history, analytics charts, report generation, and settings. The file identifies the experience as CICIDS2017 dataset replay and contains timestamps, source/destination addresses, ports, protocol, bytes sent and received, attack type, binary label, prediction, and confidence fields.

This report reviews recent machine-learning intrusion-detection research, describes the current interface and data workflow, examines data-quality and preprocessing requirements, and compares four models using published CICIDS2017 results. Published metrics are explicitly separated from descriptive metrics calculated from the supplied HTML. Because labels and predictions are embedded in the file, these descriptive metrics do not constitute a new, independently trained model evaluation. The prototype is useful for educational replay and visualization; live packet capture, reproducible model training, and production security controls remain future work.

Keywords: Intrusion Detection System, CICIDS2017, network traffic, machine learning, SGD classifier, preprocessing, cybersecurity dashboard.

Contents

ABSTRACT....2
1. Introduction....5
1.1 Background....5
1.2 Problem Statement....5
1.3 Aim and Objectives....5
1.4 Scope....5
1.5 Deployed Website....5
2. Literature Review....6
2.1 Rehman et al. (2025) — Systematic ML-IDS Review....6
2.2 Deep Learning-Driven NIDS Review (2025)....6
2.3 Classical and Hybrid Models on NSL-KDD and CICIDS2017 (2025)....6
2.4 Mishra et al. (2026) — Cross-Dataset Harmonization....6
2.5 IEEE Access Survey of Deep Learning in NIDS (2025)....6
2.6 Adversarial Robustness Review (2026)....6
2.7 Literature Comparison....6
2.8 Research Gap....7
3. Methodology....8
3.1 Development Approach....8
3.2 System Architecture....8
3.3 Workflow....8
3.4 Dataset and Implementation Qualification....8
3.5 Evaluation Plan....8
4. Data Cleaning and Preprocessing....9
4.1 Data Fields and Use....9
4.2 Cleaning Steps Recommended for a Reproducible Pipeline....9
4.3 Embedded Dataset Profile....9
4.4 Imbalance and Leakage Risks....10
5. Results and Discussion....11
5.1 Functional Test Plan....11
5.2 Descriptive Metrics from Embedded Fields....11
5.3 Discussion....12
5.4 Current Limitations....12
6. State-of-the-Art Model Comparison....12

6.1 Metrics 12
6.2 Four-Model CICIDS2017 Comparison 13
6.3 Interpretation 13
6.4 Cross-Dataset Comparison and Caveat 13
6.5 Recommended Reproducible Experiment 14
7. User Interface Development and Deployment 14
7.1 Technology Stack 14
7.2 Main Modules 14
7.2.1 Dashboard 14
7.2.2 Live Monitoring 14
7.2.3 Attack Detection 14
7.2.4 Attack History and Analytics 15
7.2.5 Reports and Settings 15
7.3 Deployment Procedure 15
7.4 Security Considerations 15
7.5 Screenshots to Insert 15
7.5.1 Login Page 15
7.5.2 Dashboard 16
7.5.3 Live Monitoring 16
7.5.4 Attack Detection 17
7.5.5 Attack History 17
7.5.6 Analytics 17
7.5.7 Reports 18
7.6 Deployment Status 18
8. Conclusion and Future Scope 19
8.1 Conclusion 19
8.2 Future Scope 19
8.3 Final Note 19
References 19
Appendix A — Embedded Dataset Summary 20

1. Introduction

1.1 Background

Computer networks support online services, cloud applications, communication, and transactions. Their connectivity also exposes systems to attacks such as denial of service, brute-force attempts, scanning, exploitation, and command-and-control activity. An Intrusion Detection System (IDS) monitors activity and identifies events that may indicate a security incident. Signature-based approaches are effective for known patterns but can struggle when traffic changes or new patterns do not match existing signatures.

Machine-learning-based IDS approaches learn patterns from labelled or unlabelled network traffic. Their performance depends on data quality, feature engineering, class balance, evaluation protocol, and whether test data represents the intended deployment environment. Recent research emphasizes that high benchmark accuracy alone does not guarantee real-world performance [1], [2], [4].

1.2 Problem Statement

Raw network event records can be difficult to interpret when presented only as tables. CyberShield addresses this usability problem by organizing recorded network-event data into an interactive replay interface with summary metrics, alerts, history, charts, and downloadable reports. The current implementation replays a dataset; it does not capture live packets from a physical network interface.

1.3 Aim and Objectives

Aim: To develop a browser-based dashboard that replays recorded network events and presents attack-related information through monitoring, detection, history, analytics, and report views.

- Create a consolidated dashboard for reviewing network events and summary statistics.
- Replay embedded network records at a configurable interval to simulate a continuous stream.
- Display IP addresses, ports, protocols, byte counts, attack types, labels, predictions, confidence, and alert details.
- Separate normal and suspicious events in the interface and display notifications where configured.
- Provide searchable and filterable attack history.
- Visualize event trends, attack distribution, severity, protocols, and source IP summaries.
- Generate and download a text-based security summary.
- Compare recent IDS research and published model metrics without presenting them as CyberShield results.
- Identify implementation limitations and recommend a reproducible evaluation path.

1.4 Scope

The report covers the supplied standalone HTML file, embedded event records, replay controls, browser-side chart rendering, filters, alert interface, and report export. It does not establish live packet capture, automatic traffic blocking, secure authentication, persistent server-side storage, or validated protection against unknown attacks.

1.5 Deployed Website

CyberShield has been deployed as a web-based application to demonstrate its cybersecurity monitoring interface. Users can access the website through a web browser to explore the dashboard, live monitoring, attack detection, attack history, analytics, reports, and settings.

Website URL: https://cyber-shield-navy-psi.vercel.app/

The deployed version provides an accessible demonstration of the project's user interface and its dataset-replay-based detection workflow.

2. Literature Review

This review focuses on recent work on machine-learning intrusion detection, data preparation, benchmark evaluation, and deployment gaps. Metrics from different datasets and protocols should not be ranked as though they were obtained in one controlled experiment.

2.1 Rehman et al. (2025) Systematic ML-IDS Review

Rehman and colleagues systematically reviewed machine-learning intrusion detection, organizing model families, benchmark datasets, and evaluation metrics. The review covers NSL-KDD, CICIDS2017, and UNSW-NB15 and highlights dataset imbalance and outdated attack patterns as limitations that can affect model evaluation [1]. This is relevant to CyberShield because its embedded data contains 9,600 normal-labelled and 400 attack-labelled records.

2.2 Deep Learning-Driven NIDS Review (2025)

The 2025 ICT Express systematic review surveys deep-learning methods for network-based IDS, including data preparation, architectures, datasets, and evaluation metrics [2]. It provides a structured view of the benefits and open challenges of deep learning in network security. For CyberShield, it motivates comparing a lightweight classical model with neural approaches, while ensuring that visual predictions are not mistaken for validated detection performance.

2.3 Classical and Hybrid Models on NSL-KDD and CICIDS2017 (2025)

The study in Algorithms compares Naïve Bayes, Logistic Regression, Linear Discriminant Analysis (LDA), and Autoencoder + Logistic Regression on NSL-KDD and CICIDS2017. It reports accuracy, precision, recall, F1-score, AUC, and false-alarm rate. On CICIDS2017 without SMOTE, Logistic Regression achieved 98.78% accuracy and 97.52% F1-score; Autoencoder + Logistic Regression achieved 98.59% accuracy and 97.15% F1-score [3]. These published values are used in Section 6.

2.4 Mishra et al. (2026) Cross-Dataset Harmonization

Mishra and colleagues proposed a harmonized evaluation framework combining NSL-KDD and CICIDS2017 through feature alignment and multi-model benchmarking. The study reports 98.0% accuracy and 97.0% F1-score for Random Forest on its harmonized dataset and emphasizes reproducibility and generalization [4]. This supports a future CyberShield workflow that tests across datasets instead of relying only on one embedded replay.

2.5 IEEE Access Survey of Deep Learning in NIDS (2025)

This survey reviews deep-learning approaches published between 2020 and 2025, including reconstruction/generative anomaly detection, temporal sequence models, convolutional classifiers, hybrid systems, and deployment-oriented methods. It identifies data quality, scalability, adversarial robustness, privacy, and interpretability as persistent challenges [5].
CyberShield could build on these themes by explaining which features influence predictions and monitoring false alarms.

2.6 Adversarial Robustness Review (2026)

A 2026 IEEE Access review examines adversarial robustness in deep-learning IDS, covering attacks, defenses, datasets, and evaluation gaps. It reports that research has emphasized white-box attacks, while black-box evaluation and certified defenses are less common [6]. This implies that even strong test metrics should not be treated as proof that an IDS is resistant to evasion.

2.7 Literature Comparison

Study	Year / scope	Methods or focus	Contribution	Implication
Rehman et al. [1]	2025; systematic review	ML models, datasets, metrics	Organizes model families and dataset limitations	Report class balance and multiple metrics
ICT Express review [2]	2025; deep-learning NIDS	DL methods and preprocessing	Surveys architectures and evaluation	Benchmark scores depend on protocol

Algorithms study [3]	2025; CICIDS2017 and NSL-KDD	NB, LR, LDA, AE+LR	Reports controlled four-model metrics	Useful for one within-study comparison
Mishra et al. [4]	2026; cross-dataset	Feature harmonization and multi-model evaluation	RF: 98.0% accuracy, 97.0% F1 on harmonized data	Generalization and reproducibility matter
IEEE Access survey [5]	2025; DL-based NIDS	CNN/RNN/transformer/hybrid themes	Summarizes deployment and interpretability gaps	Explainability and robust validation needed
IEEE Access review [6]	2026; adversarial robustness	Attacks, defenses, datasets	Identifies weaknesses in robustness evaluation	Test beyond ordinary accuracy

2.8 Research Gap

The reviewed literature identifies three recurring challenges: benchmark performance can overstate operational readiness; preprocessing and class imbalance can strongly affect results; and cross-dataset generalization, reproducibility, and adversarial robustness remain difficult. CyberShield currently provides an accessible replay and visualization interface, not a novel validated classifier. A defensible next step is to separate the dashboard from a reproducible training/evaluation pipeline.

3. Methodology

3.1 Development Approach

The project follows an iterative prototype approach: inspect the supplied event data, organize the interface into modules, implement replay and event rendering, add summary charts and filtering, and examine the label/prediction fields embedded in the file. The implementation uses HTML, CSS, and browser-side JavaScript.

3.2 System Architecture

Layer	Role
Dataset layer	JavaScript array embeds 10,000 event records in the HTML.
Replay controller	Processes records incrementally at the selected monitoring interval.
Event processing	Maps each row to display fields such as source, destination, protocol, byte counts, type, severity, confidence, and prediction.
Dashboard state	Browser-side arrays track recent events and attack events for summaries.
Presentation	HTML sections and CSS implement dashboard, monitoring, detection, history, analytics, reports, and settings.
Visualization	Custom canvas chart renderer draws charts without a thirdparty CDN.
Export	Creates a text summary and triggers a browser download.

3.3 Workflow

1. Open the standalone HTML file in a modern browser.
2. Submit the demo login form to reveal the application interface.
3. Open Live Monitoring and start dataset replay.
4. The replay controller reads the next embedded event at the configured interval.
5. The event is mapped into a display row and added to recent-event/monitoring tables.
6. Detection state, history, dashboard metrics, and charts are updated.
7. When an event is treated as an attack, the interface can display an alert and optional toast notification.
8. Search or filter history, or generate and download a text report.

3.4 Dataset and Implementation Qualification

The embedded array contains 10,000 rows with timestamp, source, destination, ports, protocol, sent/received bytes, type, binary label, prediction, and confidence fields. The UI describes the source as CICIDS2017 dataset replay. However, the event names include categories such as brute-force, port-scan, SQL injection, XSS, credential stuffing, DDoS, command injection, exploit attempt, and C2. Row-level provenance and any transformations from the original CICIDS2017 data have not been independently verified.

The detection panel refers to an incremental SGD classifier and prequential evaluation, but the records already contain label, prediction, and confidence fields. The browser uses these embedded fields for display and metric updates. Therefore, this report does not claim that the supplied browser file independently trains a model from raw data and evaluates it on a held-out test set.

3.5 Evaluation Plan

Functional evaluation covers navigation, replay start/stop/reset, event rendering, filters, alerts, chart refresh, and report download. Descriptive analysis compares the embedded labels and predictions. Published model scores are discussed separately in Section 6.

4. Data Cleaning and Preprocessing

4.1 Data Fields and Use

Field	Meaning	Use / preprocessing
timestamp	Recorded event time	Parsed for display; original string may be fallback.
source / dest	IP addresses	Rendered as strings in tables and alerts.
srcPort / dstPort	Ports	Displayed as numeric event attributes.
protocol	Protocol	Grouped for protocol analytics.
sent / received	Bytes sent/received	Used for total byte size and derived flow display.
type	Event category	Displayed in alert/history views.
label	Binary label 0/1	Compared with prediction for descriptive metrics.
prediction	Embedded prediction 0/1	Used by the dashboard metric calculation.
confidence	Embedded confidence	Converted to a percentage for display.

4.2 Cleaning Steps Recommended for a Reproducible Pipeline

Check required fields for missing timestamps, addresses, protocols, ports, byte counts, labels, and predictions.

Normalize timestamps and timezone; document invalid or ambiguous date strings.

Validate numeric fields; reject negative byte counts and invalid port ranges.

Normalize protocol and attack-label spelling/case.

Remove duplicates using a documented flow/time key where appropriate.

Inspect identifiers and metadata for possible leakage or memorization.

Fit encoders, imputers, and scalers on training data only, then apply them to validation/test data.

Use stratified splits where suitable and time-based/group-aware splits where needed to reduce leakage.

Report class distribution and use precision, recall, F1, false-positive rate, and AUC in addition to accuracy.

4.3 Embedded Dataset Profile

The embedded array has 10,000 records. Its binary label field contains 9,600 normal-labelled records (96.00%) and 400 attack-labelled records (4.00%). The most common protocol is TCP with 7,765 records.

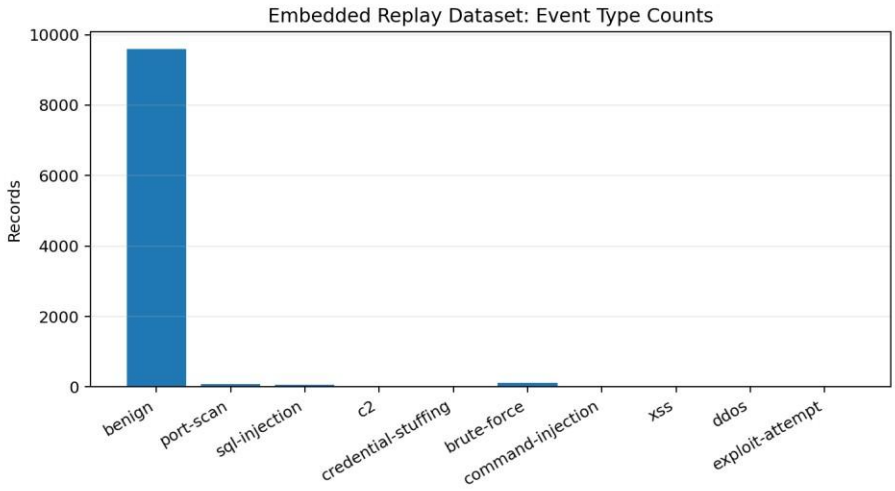


Figure 1. Distribution of event types in the embedded replay data.

4.4 Imbalance and Leakage Risks

The labels are imbalanced. A model that predicts the majority class may achieve deceptively high accuracy while missing attacks. Further, embedded predictions do not prove how or on which split a model was trained. A rigorous future pipeline should load raw source files, record cleaning steps, train on a training split, and calculate metrics on a held-out set.

5. Results and Discussion

5.1 Functional Test Plan

Test	Action	Expected result	Status basis
T01	Submit demo login	Application appears	Handler present; verify in browser
T02	Navigate sidebar views	Selected section and title change	Navigation handlers present
T03	Start replay	Rows appear incrementally	Replay controls present
T04	Stop/reset replay	Timer stops and replay state resets	Stop/reset handlers present
T05	Search/filter history	Only matching events remain	Filter listeners present
T06	Process attack record	Attack details/notification may appear	Driven by embedded event fields
T07	Open analytics	Charts reflect current state	Canvas renderer present
T08	Generate/download report	Text summary downloads	Blob/download handler present
T09	Change interval	Replay interval changes	Settings handler present
T10	Capture actual network packets	Live traffic is collected	Not implemented

5.2 Descriptive Metrics from Embedded Fields

Comparing all 10,000 embedded label/prediction pairs yields TP=33, TN=8960, FP=640, FN=367. The descriptive accuracy is 89.93%, precision 4.90%, recall 8.25%, F1-score 6.15%, and false-positive rate 6.67%. These numbers were calculated from the existing fields in the supplied HTML; they are not a result of retraining the model.

Metric	Value	Meaning
True positives (TP)	33	Attack-labelled record predicted as attack
True negatives (TN)	8960	Normal-labelled record predicted as normal
False positives (FP)	640	Normal-labelled record predicted as attack
False negatives (FN)	367	Attack-labelled record predicted as normal
Accuracy	89.93%	All correct predictions / all records
Precision	4.90%	Reliability of predicted attack events
Recall	8.25%	Share of attack-labelled events detected
F1-score	6.15%	Harmonic mean of precision and recall
False-positive rate	6.67%	Share of normal records incorrectly flagged

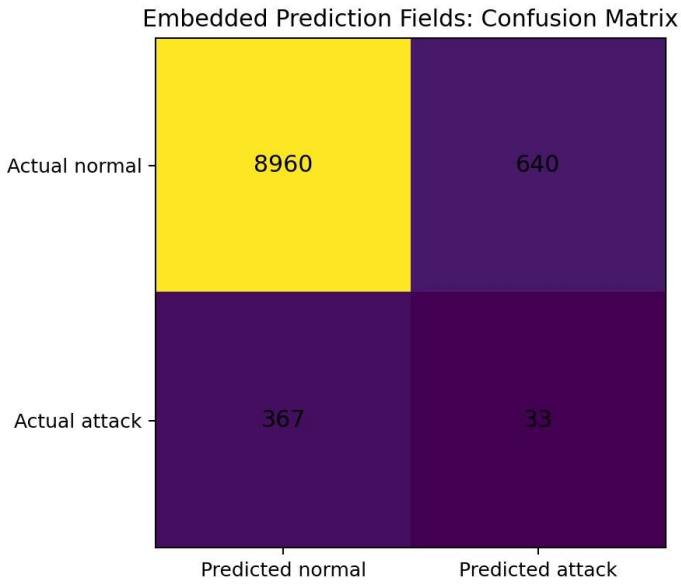


Figure 2. Confusion matrix calculated from embedded label and prediction fields.

5.3 Discussion

The descriptive results illustrate why accuracy alone is insufficient. The embedded accuracy is close to 90%, but precision and recall calculated from the same fields are low. Under the binary labels in the file, many attack-labelled records are missed and many normal-labelled records are flagged. Before making a model-performance claim, the dataset provenance, label mapping, prediction-generation process, confidence calculation, and evaluation split must be verified. Low recall is especially important because missed attacks can reduce the usefulness of a security alerting system.

The interface is still useful as an educational visualization prototype because it demonstrates replay, event tables, alerts, filters, charts, and report export. The highest-priority technical improvement is a reproducible model pipeline rather than additional visual decoration.

5.4 Current Limitations

No raw network packet capture is implemented; the system replays embedded rows.

Row-level provenance against the original CICIDS2017 files is not verified.

Embedded predictions do not establish independent retraining or held-out testing.

The login screen is a demo gate, not secure authentication.

Replay state is browser-side; no persistent server-side event database is included.

The system does not block traffic or perform incident response actions.

6. State-of-the-Art Model Comparison

6.1 Metrics

Accuracy measures the fraction of correct predictions. Precision measures how many predicted attacks are actually attacklabelled. Recall measures how many attack-labelled records are detected. F1-score balances precision and recall. Falsepositive rate measures normal records incorrectly flagged. AUC summarizes ranking quality across thresholds. In security systems, recall and false-positive rate matter because missed attacks and excessive false alarms both have operational costs.

Accuracy = (TP + TN) / total; Precision = TP / (TP + FP); Recall = TP / (TP + FN); F1 = 2 × Precision × Recall / (Precision + Recall).

6.2 Four-Model CICIDS2017 Comparison

The following table reproduces the no-SMOTE CICIDS2017 results reported in the 2025 deterministic comparison study [3]. The four models were evaluated within the same paper and dataset setting. These are published literature results, not CyberShield measurements.

Model	Accuracy %	Precision %	Recall %	F1 %	FAR %	AUC
Naïve Bayes	85.65	63.64	97.42	76.99	18.20	0.9711
Logistic Regression	98.78	97.22	97.83	97.52	0.91	0.9961
LDA	97.84	94.26	97.15	95.69	1.93	0.9923
Autoencoder + Logistic Regression	98.59	96.91	97.38	97.15	1.01	0.9960

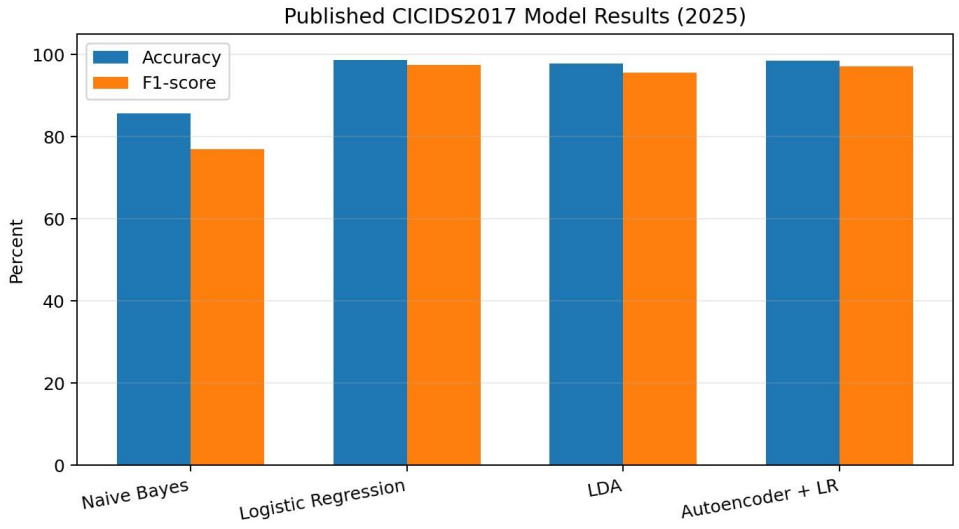


Figure 3. Published accuracy and F1-score for four models on CICIDS2017 [3].

6.3 Interpretation

In the cited experiment, Logistic Regression has the highest reported accuracy and F1-score among the four models. Autoencoder + Logistic Regression is close behind, using a learned representation followed by a linear classifier. LDA is also strong. Naïve Bayes has high recall in this table but lower precision and a much higher false-alarm rate, showing that a single metric can hide operational weaknesses [3].

6.4 Cross-Dataset Comparison and Caveat

Study / dataset	Model	Accuracy	F1-score	Interpretation
CICIDS2017, 2025 [3]	Logistic Regression	98.78%	97.52%	Within-study benchmark; without SMOTE
CICIDS2017, 2025 [3]	Autoencoder + LR	98.59%	97.15%	Hybrid representation and classifier
Harmonized NSL-KDD + CICIDS2017, 2026 [4]	Random Forest	98.0%	97.0%	Different harmonization/evaluation protocol

CyberShield embedded fields	Existing prediction field	89.93%	6.15%	Descriptive calculation; no independent retraining

The CyberShield row is not ranked against the published studies because the data-generation and evaluation protocols differ. It identifies a validation gap rather than proving that one algorithm is intrinsically better.

6.5 Recommended Reproducible Experiment

1. Use original documented CICIDS2017 files and publish the exact subset and label mapping.
2. Remove duplicates and invalid rows; document missing-value handling and categorical encoding.
3. Split data before fitting preprocessing; use group-aware or time-aware evaluation where appropriate.
4. Train at least four models under one common protocol: Logistic Regression, Random Forest, Gradient Boosting/XGBoost, and MLP or 1D-CNN.
5. Report accuracy, precision, recall, F1, false-positive rate, ROC-AUC, PR-AUC, inference time, and confusion matrices.
6. Fix random seeds and report variation for repeated experiments or cross-validation.
7. Connect the selected validated model to the UI and display its version and evaluation date.

7. User Interface Development and Deployment

7.1 Technology Stack

Technology	Purpose
HTML5	Structure for login, navigation, cards, tables, charts, settings, and report areas.
CSS3	Dark cybersecurity theme, responsive layout, cards, colors, and status states.
JavaScript	Navigation, replay state, event processing, alerts, metrics, filters, chart updates, and export.
Canvas API	Draws charts directly in the browser without a remote chart CDN.
Blob and browser download	Creates and downloads a text report.
Embedded JavaScript dataset	Provides 10,000 preloaded event records for demonstration.

7.2 Main Modules

7.2.1 Dashboard

Presents the security overview, summary metrics, traffic trends, attack distribution, and recent alerts.

7.2.2 Live Monitoring

Provides Start Dataset Replay, Stop Monitoring, and Reset Replay controls. Its table displays timestamp, source/destination IP, protocol, ports, bytes, packet size, derived flow information, and status. The stream is recorded data replay, not live packet capture.

7.2.3 Attack Detection

Displays the latest traffic classification and confidence, and can show attack type, source, destination, protocol, severity, and confidence. The pipeline description refers to preprocessing, feature extraction, incremental SGD prediction plus dataset label, classification, and alert updates. The prediction-generation process should be independently documented before this is treated as a validated detector.

7.2.4 Attack History and Analytics

History supports search and filters by attack type, severity, and status. Analytics summarizes event trends, attack categories, severity, top source IPs, and protocols using canvas-based charts.

7.2.5 Reports and Settings

Reports creates a concise text summary and downloads it as a .txt file. Settings expose replay interval, alert-confidence threshold, theme selection, notification preference, and system status. The effect of the threshold setting should be confirmed during manual testing.

7.3 Deployment Procedure

1. Keep a backup of the supplied HTML file.
2. Open the file in a current desktop browser for offline demonstration.
3. For a hosted demonstration, publish the static HTML through a hosting service that supports HTTPS and test the deployed build.
4. Check the browser console and verify replay, filters, charts, and download behavior after deployment.
5. Do not enter real credentials or sensitive production network logs into the demo.
6. If a backend is added, serve normalized events and validated predictions through a protected API; never expose secrets in client-side JavaScript.

7.4 Security Considerations

The login form only hides or reveals the application; it does not verify credentials against a server. It must be described as a demo login screen, not secure authentication. The supplied HTML contains no backend database or live packet sensor. Production use would require authentication, authorization, HTTPS, input validation, server-side logging, retention controls, and safe handling of IP addresses.

7.5 Screenshots to Insert

7.5.1 Login Page

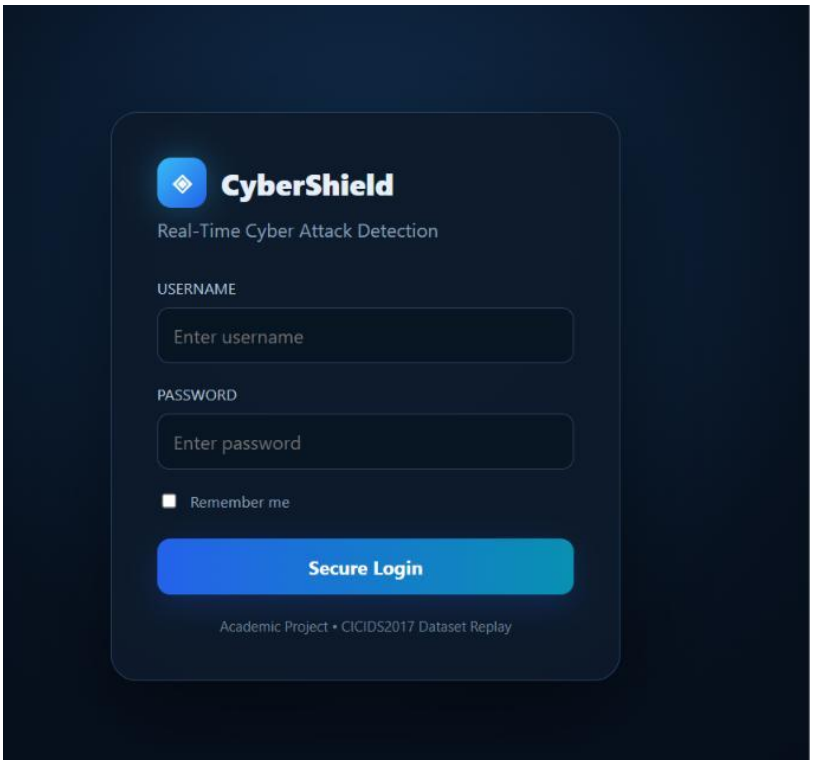


Figure 4. CyberShield login screen.

7.5.2 Dashboard

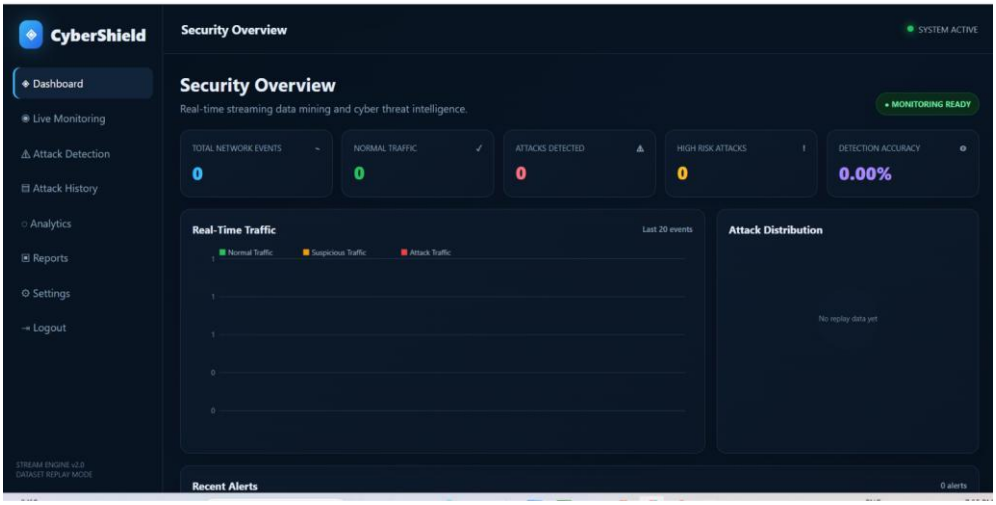


Figure 5. Security overview dashboard before replay data is populated.

7.5.3 Live Monitoring

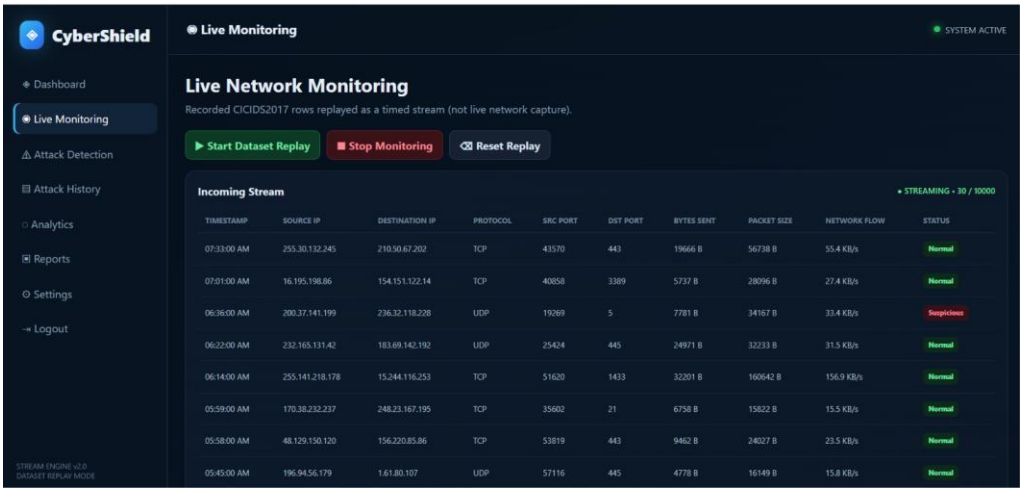


Figure 6.Recorded network events displayed during dataset replay.

7.5.4 Attack Detection

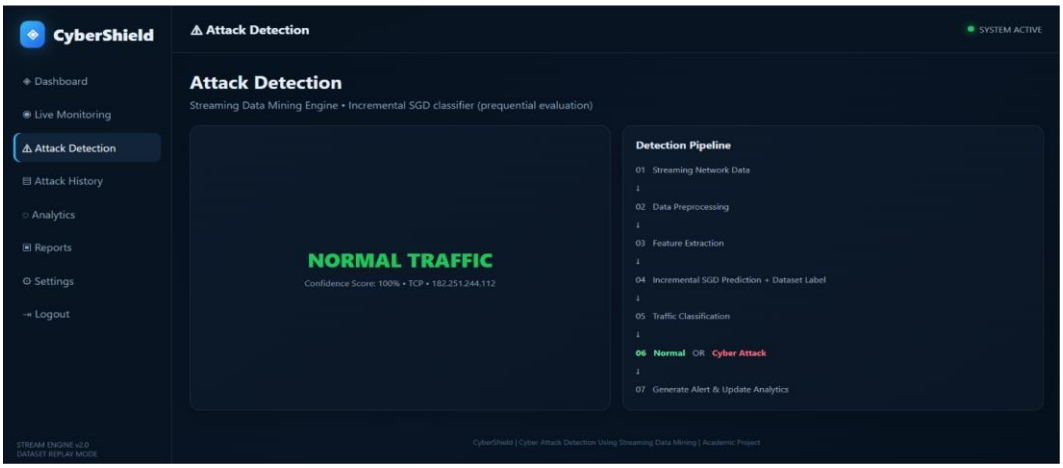


Figure 7. Attack Detection panel with event details.

7.5.5 Attack History

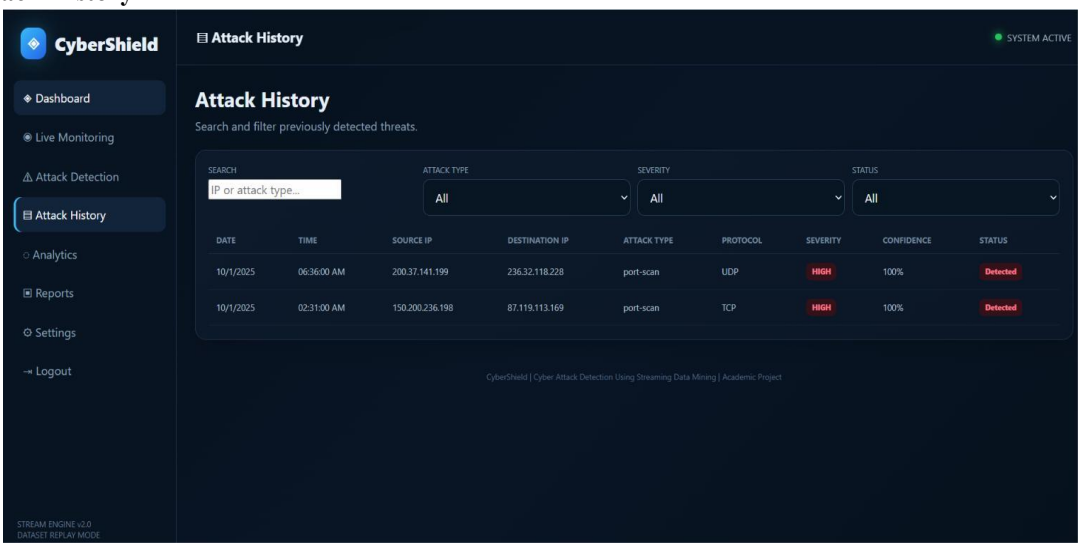


Figure 8. Attack History with search and filters.

7.5.6 Analytics

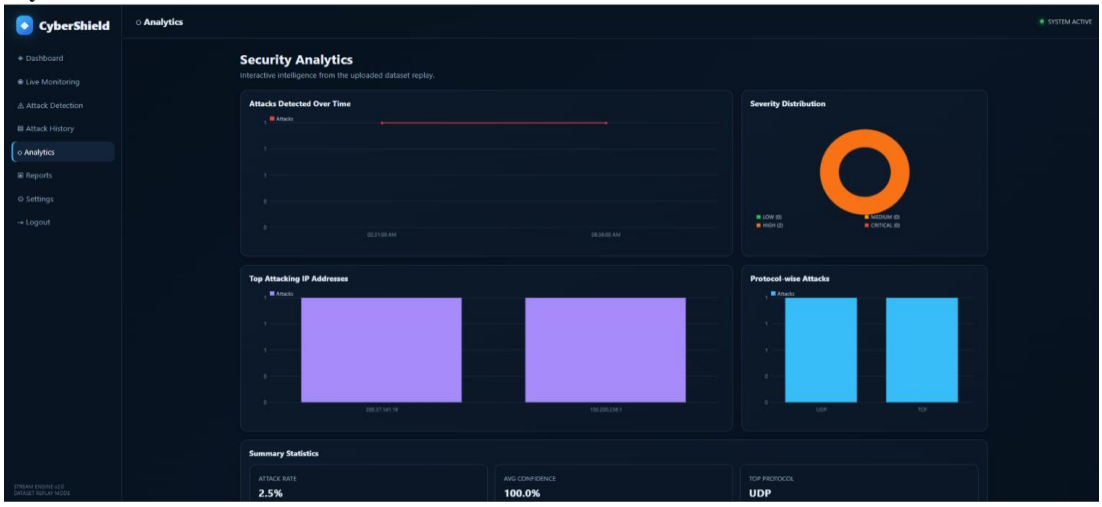


Figure 9. Analytics charts.

7.5.7 Reports

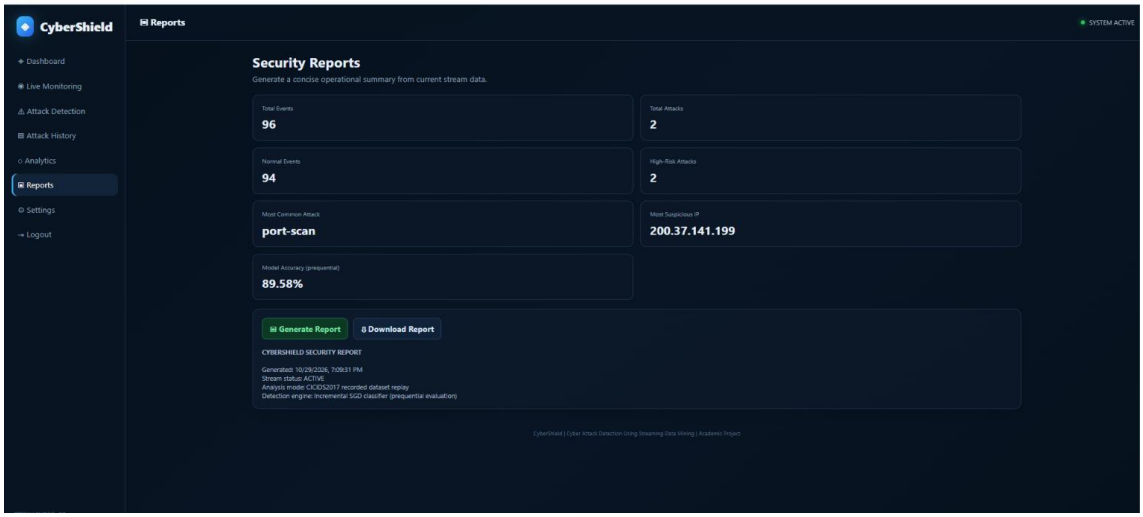


Figure 10. Generated Security Report.

7.6 Deployment Status

https://github.com/Trupti-Avhadkar/CyberShield_

8. Conclusion and Future Scope

8.1 Conclusion

CyberShield demonstrates how a browser-based interface can organize recorded network-event data into a cybersecurity dashboard. Its implemented features include a demo login, navigation, replay controls, monitoring tables, attack alerts, history filters, canvas charts, settings, and text-report export. The embedded array contains 10,000 records and supports demonstration and descriptive analysis.

The data review identifies a validation gap: label and prediction fields are embedded in the HTML, and the descriptive metrics show low precision and recall despite near-90% accuracy. These figures should not be presented as the results of a newly trained model. Recent literature shows the importance of preprocessing, class-aware metrics, reproducibility, cross-dataset testing, and robustness analysis.

8.2 Future Scope

Replace the embedded array with a documented data loader and reproducible model training/evaluation pipeline.

Train and compare Logistic Regression, Random Forest, Gradient Boosting/XGBoost, and a neural model using one common protocol.

Connect a validated model through a backend API or controlled stream service.

Integrate lawful live flow/packet telemetry with consent and privacy controls.

Add model versioning, calibrated confidence, feature explanations, and data-drift monitoring.

Store alerts in a protected database and add CSV/PDF export and audit logs.

Implement real authentication and role-based access if the system is extended beyond an academic demo.

Tune thresholds to balance recall and false alarms.

Evaluate cross-dataset generalization and adversarial robustness before claiming operational readiness.

8.3 Final Note

CyberShield should currently be presented as an educational dataset-replay and visualization prototype for network intrusion detection. Claims of production-grade real-time detection should be made only after integration with live telemetry and a reproducibly evaluated model.

References

[1] H. M. R. Ur Rehman, S. Liaquat, M. J. Gul, M. Z. Jhandir, D. Gavilanes, M. M. Vergara, et al., "A systematic literature study of machine learning techniques based intrusion detection: datasets, models, challenges, and future directions," Journal of Big Data, vol. 12, article 264, 2025. https://doi.org/10.1186/s40537-025-01323-2

[2] "Deep learning-driven methods for network-based intrusion detection systems: A systematic review," ICT Express, vol. 11, no. 1, pp. 181–215, 2025. https://doi.org/10.1016/j.icte.2025.01.005

[3] "A Deterministic Comparison of Classical Machine Learning and Hybrid Deep Representation Models for Intrusion Detection on NSL-KDD and CICIDS2017," Algorithms, vol. 18, no. 12, article 749, 2025. https://doi.org/10.3390/a18120749

[4] S. Mishra, N. S. Alshammari, H. Hussain, and R. A. Alfahidah, "A cross-dataset harmonized intrusion detection framework with statistically validated multi-model learning," PLOS ONE, vol. 21, no. 4, e0346982, 2026. https://doi.org/10.1371/journal.pone.0346982

[5] "A Survey on the Applications of Deep Learning in Network Intrusion Detection Systems to Enhance Network Security," IEEE Access, vol. 13, 2025. https://doi.org/10.1109/ACCESS.2025.3624952

[6] "Adversarial Robustness of Deep Learning-Based Intrusion Detection Systems: A Systematic Literature Review of Attacks, Defenses, Datasets, and Open Challenges," IEEE Access, vol. 14, pp. 149555–149591, 2026. https://doi.org/10.1109/ACCESS.2026.3733985

[7] National Institute of Standards and Technology, “Guide to Intrusion Detection and Prevention Systems (IDPS),” NIST Special Publication 800-94, 2007. https://csrc.nist.gov/pubs/sp/800/94/final

[8] Canadian Institute for Cybersecurity, “CICIDS2017 Dataset,” University of New Brunswick.
https://www.unb.ca/cic/datasets/ids-2017.html

Appendix A Embedded Dataset Summary

Attribute	**Observed value**
Record count	10,000
Normal-labelled records	9,600 (96.00%)
Attack-labelled records	400 (4.00%)
Protocols	TCP: 7,765, UDP: 1,910, ICMP: 325
Most common event type	benign (9,600)
Provenance caveat	The file describes CICIDS2017 replay; row-level provenance is not independently verified.